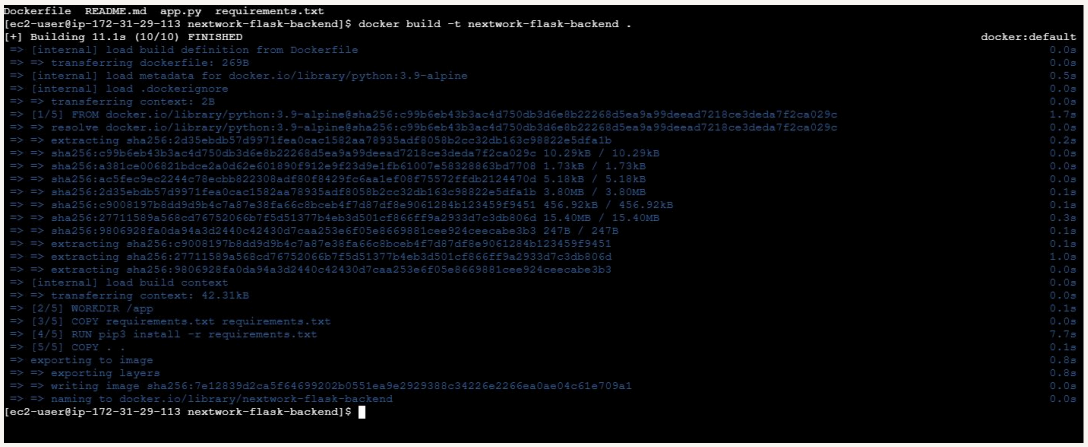
Set Up Kubernetes Deployment

Mohammed Dawoud Mota



Introducing Today's Project!

Today I am preparing the backend of an application so it is ready for deployment on a Kubernetes cluster running in Amazon EKS. My goal is to take the backend code that was shared through GitHub, clone it into my EC2 instance, build a Docker image from that code, and then push that image into an Amazon ECR repository where Kubernetes can pull it during deployment. As I work through the project I will set up my EC2 environment, install the required tools, fix any configuration or permission issues, and make sure the backend is packaged in a clean and consistent way. By the end of this process, I will have a fully built and stored container image that is ready for Kubernetes to use when the application is deployed.

Tools and concepts

In this project I used a combination of AWS services and development tools to build, package, and prepare my backend application for deployment. I started by launching an EC2 instance and setting it up as my working environment. From there I installed eksctl so I could create and manage my EKS cluster. After that I configured Git and cloned my backend repository from GitHub so the code was available on my instance. Once the code was in place I installed Docker, fixed the permission issue, and built a container image for the backend. I then created an ECR repository, authenticated Docker, tagged my image, and pushed it to ECR so Kubernetes would have a reliable place to pull from during deployment. All of these tools and steps worked together to prepare my backend for running inside an EKS cluster.

Project reflection

This project took me less than an hour to complete. My favourite part was seeing how the Kubernetes clusters were being deployed.

One thing I learned from this project was how to push a Docker image to Amazon ECR and make it available for Kubernetes. I've built Docker images before, but actually authenticating Docker to ECR, tagging the image correctly, and pushing it to a private AWS registry was new to me. I also learned how important it is to set the AWS Region when using eksctl, because without it the tool can't locate or manage the cluster. Overall, this project helped me understand the full workflow of taking backend code, packaging it into a container, storing it in ECR, and preparing it for deployment inside an EKS cluster.

What I'm deploying

To set up my Kubernetes cluster I launched a new EC2 instance in my AWS account and connected to it using EC2 Instance Connect. Once I had access to the instance I installed eksctl, which is the tool that simplifies creating and managing EKS clusters. I then created an IAM role with AdministratorAccess and attached it to my EC2 instance so it had the permissions required to interact with AWS services. After the role was attached and my environment was ready I used eksctl to create the EKS cluster. This process allowed AWS to automatically provision all the networking components, the control plane, and the node group that make up the cluster. By the end of the setup I had a fully functioning Kubernetes cluster that I could use to deploy applications.

I'm deploying an app's backend

I got the backend code by cloning the GitHub repository directly into my EC2 instance. After installing and configuring Git with my name and email, I used the git clone command to pull down the entire nextwork flask backend repository. This gave me a complete local copy of all the backend files including the application code, the Dockerfile, and the requirements needed to build the container image. Cloning the repository ensured that I had the exact version of the backend that my team member pushed to GitHub, allowing me to continue the deployment process with a clean and consistent codebase.

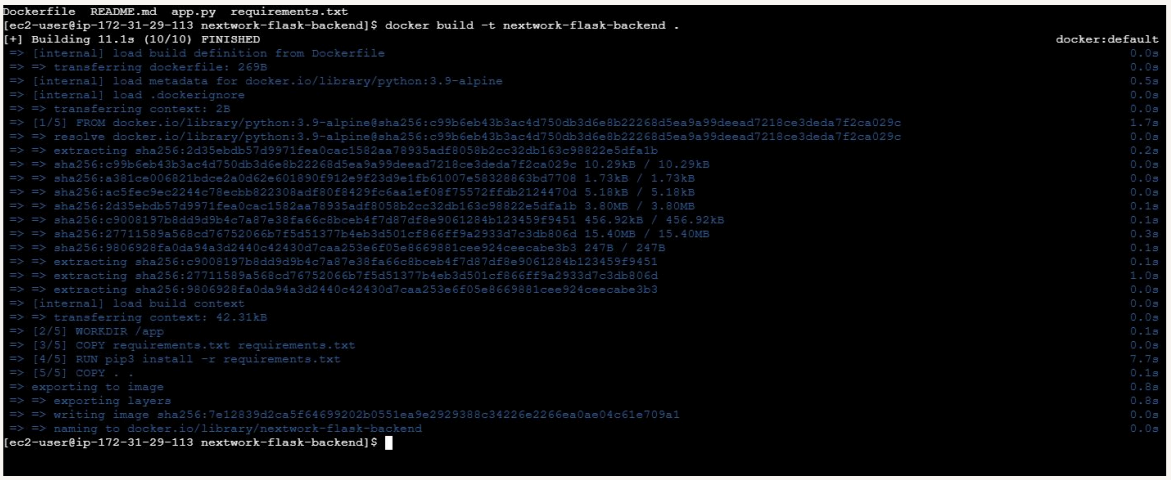

[ec2-user@ip-172-31-29-113 ~]$ ls
nextwork-flask-backend

Building a container image

I am building a container image because Kubernetes needs a packaged and consistent version of my backend application that it can pull and run inside the cluster. By creating a Docker image I am bundling the backend code, its dependencies, and its runtime configuration into a single portable unit. This ensures the application behaves the same no matter where it is deployed. It also allows EKS to scale the application easily since Kubernetes can launch multiple identical containers from the same image whenever needed. Building this image is a critical step in preparing the backend for a smooth and reliable deployment.

When I tried to build the Docker image I ran into a permissions error because the ec2 user did not have the rights to communicate with the Docker engine. Docker was installed under the root account, which meant only the root user could run Docker commands. Since I was logged in as ec2 user, any attempt to build the image without sudo resulted in Docker blocking the command. To fix this I added the ec2 user to the Docker group so it could run Docker commands normally and then restarted the session to apply the new permissions.

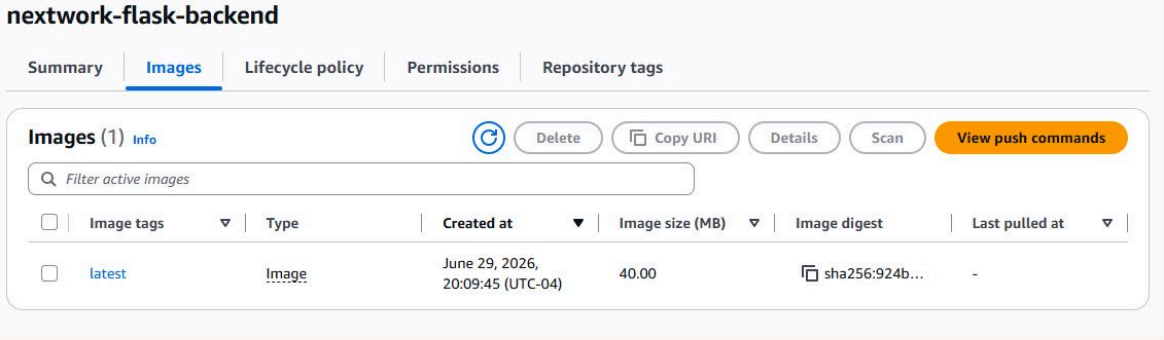
I resolved the permissions error by giving my ec2 user the ability to run Docker commands directly. Since Docker was installed under the root account, the ec2 user did not have permission to communicate with the Docker engine, which caused the build command to fail unless I used sudo. To fix this I added the ec2 user to the Docker group, which is the group that grants access to Docker on Linux systems. After running the usermod command I restarted my terminal session so the new group membership would take effect. Once the session refreshed my ec2 user had the correct permissions and I was able to build the Docker image without any issues.


Dockerfile README.md app.py requirements.txt
[ec2-user@ip-172-31-29-113 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
[+] Building 11.1s (10/10) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 269B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-alpine 0.5s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 1.7s
=> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 0.0s
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB 0.0s
=> => sha256:a381ce006821bdce2a0d62e601890f912e9f23d9e1fb61007e58328863bd7708 1.73kB / 1.73kB 0.0s
=> => sha256:ac5fec9ec2244c78ecbb822308adf80f8429fc6aa1ef08f75572ffdb2124470d 5.18kB / 5.18kB 0.0s
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB 0.1s
=> => sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB 0.3s
=> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B 0.1s
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 0.1s
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 1.0s
=> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 42.31kB 0.0s
=> [2/5] WORKDIR /app 0.1s
=> [3/5] COPY requirements.txt requirements.txt 0.0s
=> [4/5] RUN pip3 install -r requirements.txt 7.7s
=> [5/5] COPY . . 0.1s
=> exporting to image 0.8s
=> => exporting layers 0.8s
=> => writing image sha256:7e12839d2ca5f64699202b0551ea9e2929388c34226e2266ea0ae04c61e709a1 0.0s
=> => naming to docker.io/library/nextwork-flask-backend 0.0s
[ec2-user@ip-172-31-29-113 nextwork-flask-backend]$

Container Registry

I am using Amazon ECR because it provides a secure and reliable place to store the Docker image that I built for my backend application. Once the image is in ECR, my Kubernetes cluster in EKS can easily pull it during deployment without any extra configuration or manual steps. ECR integrates smoothly with other AWS services, which makes the entire workflow more efficient and reduces the chances of running into authentication or compatibility issues. By pushing my image to ECR, I ensure that my backend is stored in a centralized, versioned, and highly available registry that supports consistent and scalable deployments.

Using a container registry gives me a reliable and centralized place to store the Docker image for my backend application. Instead of keeping the image only on my EC2 instance, the registry provides a secure and highly available location where Kubernetes can pull the image whenever it needs to deploy or scale the application. This ensures consistency because every container launched in the cluster uses the exact same image. It also simplifies version control since I can push updated images without manually distributing them across different machines. Overall, a container registry makes deployments cleaner, more organized, and much easier to manage at scale.



The place to learn & showcase your skills

Check out nextwork.ai for more projects